

Softwarequalität

Hausarbeit

Studiengang Informatik

Duale Hochschule Baden-Württemberg Karlsruhe

Etienne Luke Josef Bader

Eingereicht am: 19.05.2026

Matrikelnummer, Kurs: 9578543, TINF23B2

Betreuer an der DHBW: Dennis Kube, Jonathan Schwarzenböck

Inhalt

1	Einleitung	1
1.1	Projektbeschreibung	1
1.2	Ziel der Arbeit	1
2	Grundlagen	3
2.1	Clean Architecture	3
2.1.1	Motivation und Ziel	3
2.1.2	Die Dependency Rule	4
2.1.3	Schichtenaufbau	4
2.1.4	Innere Schichten definieren Interfaces	5
2.2	Softwarequalität nach ISO/IEC 25010	5
3	Architektur des Projekts	7
3.1	Schichtenstruktur	7
3.2	Stärken der Umsetzung	7
3.3	Der kritische Befund	7
4	Analyse und Bewertung	8
4.1	Warum ist das kritisch?	8
4.2	Bewertung mit FMEA/RPZ	8
4.3	Widerspruch zur eigenen Dokumentation	8
5	Optimierungsmaßnahmen	9
5.1	Repository-Zugriffe in Controller eliminieren	9
5.2	Service-Annotation verschieben	9
5.3	Erwartete Auswirkungen im PDCA-Rahmen	9
6	Fazit	10
A	Literatur	11
B	Glossar	12



Einleitung

1.1 Projektbeschreibung

Das Sportwettbewerbs-Managementtool ist eine im Rahmen der Vorlesung Advanced Software Engineering an der Dualen Hochschule Baden-Württemberg Karlsruhe entwickelte Anwendung. Sie unterstützt Organisatorinnen und Organisatoren bei der Planung und Durchführung von Sportwettbewerben und bietet Funktionen zur Verwaltung von Personen, Teams, Wettkämpfen, Spielplänen, Lageplänen und Tickets. Das System besteht aus einem Java-Backend auf Basis des Spring-Boot-Frameworks sowie einem React-Frontend und wird über eine REST-API angesteuert.

Ein zentrales Merkmal des Projekts ist die bewusste Entscheidung für eine Clean Architecture, auch bekannt als Onion Architecture. Diese gliedert das System in vier klar voneinander getrennte Schichten: Domain, Application, Adapter und Plugins. Ziel dieser Architektur ist es, die fachliche Kernlogik von technischen Details wie Frameworks oder Persistenzmechanismen zu entkoppeln und so langfristige Wartbarkeit und Erweiterbarkeit sicherzustellen.

1.2 Ziel der Arbeit

Im Mittelpunkt dieser Hausarbeit steht die Frage, inwieweit die gewählte Architektur im Sinne des Softwarequalitätsmanagements tatsächlich konsequent umgesetzt wurde. Als Bewertungsmaßstab dienen zwei Teilmerkmale des Qualitätsmerkmals Maintainability aus dem internationalen Standard ISO/IEC 25010: Modularity und Modifiability. Modularity beschreibt, in welchem Maße ein System aus unabhängigen Komponenten besteht, sodass Änderungen an einer Komponente möglichst geringe Auswirkungen auf andere haben. Modifiability

beschreibt, wie effektiv das System verändert werden kann, ohne dabei unbeabsichtigte Seiteneffekte in anderen Teilen zu erzeugen.

Beide Teilmerkmale sind unmittelbar mit dem Anspruch der Clean Architecture verknüpft und eignen sich daher als konkreter, messbarer Maßstab zur Bewertung ihrer Umsetzung. Die Analyse konzentriert sich auf den `CompetitionController` als repräsentatives Fallbeispiel, an dem sich eine systematische Abweichung vom eigenen Architekturanspruch belegen lässt. Daraus werden anschließend begründete Optimierungsmaßnahmen abgeleitet.



Grundlagen

2.1 Clean Architecture

2.1.1 Motivation und Ziel

Moderne Softwaresysteme unterliegen einem beständigen Wandel. Technologien, Frameworks und Abhängigkeiten veralten, werden ersetzt oder weiterentwickelt – häufig in einem Rhythmus von wenigen Jahren [1, S. 2]. Eine Architektur, die eng an konkrete Technologien geknüpft ist, altert zwangsläufig mit diesen zusammen und erschwert spätere Anpassungen erheblich.

Die Clean Architecture begegnet diesem Problem durch eine klare strukturelle Trennung von langlebigem und kurzlebigem Code. Ihr zentrales Ziel ist es, einen technologieunabhängigen Kern zu schaffen, der sämtliche fachliche Logik enthält, und alle technischen Details – Datenbanken, Frameworks, Benutzeroberflächen – als austauschbare Randkomponenten zu behandeln [1, S. 9]. Die Metapher der Zwiebel (Onion Architecture) beschreibt diesen Aufbau treffend: Der Kern ist langlebig und stabil, die äußeren Schichten sind kurzlebiger und leichter ersetzbar [1, S. 9].

Das angestrebte Ergebnis ist ein System, in dem Technologieentscheidungen spät getroffen oder nachträglich revidiert werden können, ohne den Anwendungskern zu berühren [1, S. 8]. Robert C. Martin, der die Clean Architecture maßgeblich geprägt hat, fasst dies so zusammen: Eine gute Architektur maximiert die Anzahl der Entscheidungen, die noch nicht getroffen werden müssen [2, S. 141].

2.1.2 Die Dependency Rule

Das zentrale Prinzip der Clean Architecture ist die Dependency Rule. Sie besagt, dass Abhängigkeiten zwischen Systemteilen ausschließlich von außen nach innen zeigen dürfen [1, S. 11]. Dabei ist zwischen zwei Arten von Pfeilen zu unterscheiden: Abhängigkeitspfeile (Compile-Time Dependencies) zeigen an, welchen Code eine Klasse direkt referenziert und ohne den sie nicht kompilierbar wäre. Aufrufpfeile (Runtime Dependencies) beschreiben hingegen, welche Klasse zur Laufzeit welche andere aufruft — diese können in beide Richtungen verlaufen [1, S. 12].

Die Dependency Rule betrifft ausschließlich die Abhängigkeitspfeile: Innere Schichten dürfen äußere Schichten weder kennen noch referenzieren. Eine Verletzung dieser Regel — etwa wenn eine innere Schicht eine Klasse einer äußeren Schicht importiert — untergräbt die gesamte Architektur, da Änderungen an der äußeren Schicht dann unmittelbar Auswirkungen auf den eigentlich stabilen Kern haben [2, S. 203].

2.1.3 Schichtenaufbau

Die Clean Architecture gliedert ein System typischerweise in vier Schichten [1, S. 14]:

Die **Domain-Schicht** bildet den innersten Kern. Sie enthält die zentralen Geschäftsobjekte (Entities) sowie die organisationsweit gültigen Geschäftsregeln, die unabhängig davon existieren, ob sie in einer konkreten Anwendung nachmodelliert wurden [1, S. 18–19]. Diese Schicht sollte sich am seltensten ändern und ist vollständig immun gegen Änderungen an Infrastrukturdetails wie Anzeige, Transport oder Speicherung [1, S. 18].

Die **Application-Schicht** enthält die anwendungsspezifischen Anwendungsfälle (Use Cases). Sie steuert den Daten- und Aktionsfluss zwischen den Entities und implementiert Regeln, die nur für den konkreten Anwendungsfall gelten — beispielsweise Workflows oder Validierungsschritte [1, S. 23–24]. Diese Schicht ist isoliert von Änderungen an Datenbank oder Benutzeroberfläche, reagiert aber auf veränderte Anforderungen [1, S. 25].

Die **Adapter-Schicht** vermittelt zwischen der Anwendungslogik und der Außenwelt. Sie übernimmt Formatkonvertierungen, sodass Daten aus der Anwendungsschicht in ein für die Plugins geeignetes Format überführt werden und umgekehrt [1, S. 30]. Ihr Ziel ist die vollständige Entkopplung von innen und außen — kein SQL in der Anwendung selbst, keine Renderlogik im Kern [1, S. 31].

Die **Plugin-Schicht** ist die äußerste Schicht und enthält Frameworks, Datentransportmittel und andere technische Werkzeuge wie Datenbank, Web und Benutzeroberfläche [1, S. 37]. Diese Schicht soll hauptsächlich Delegationscode enthalten, der Aufrufe an die Adapter

weiterleitet. Auf keinen Fall darf sie Anwendungslogik enthalten — alle Entscheidungen sollen bereits in den inneren Schichten gefallen sein [1, S. 37–38].

Diese Schichtenstruktur spiegelt sich direkt in den erwarteten Lebenszyklen wider: Während Domain-Code Jahrzehnte Bestand haben kann, veralten Plugin-Code und Frameworks mitunter innerhalb von Wochen bis Monaten [1, S. 50].

2.1.4 Innere Schichten definieren Interfaces

Ein wesentliches Umsetzungsmittel der Dependency Rule ist die konsequente Nutzung von Interfaces: Innere Schichten definieren Schnittstellen, äußere Schichten implementieren diese [1, S. 16]. So kann die Domain-Schicht beispielsweise ein Repository-Interface definieren, ohne zu wissen, ob die konkrete Implementierung eine Datei, eine relationale Datenbank oder einen Webservice verwendet. Die Plugin-Schicht implementiert dieses Interface und koppelt sich damit an die innere Schicht — nicht umgekehrt. Dieses Muster wird auch als Dependency Inversion Principle bezeichnet und ist eine der tragenden Säulen der Clean Architecture [2, S. 91].

2.2 Softwarequalität nach ISO/IEC 25010

ISO/IEC 25010 ist der internationale Standard zur Beschreibung und Bewertung von Softwarequalität. Er definiert ein hierarchisches Qualitätsmodell, das Qualitätsmerkmale in Hauptmerkmale und Teilmerkmale untergliedert [3, Abschn. 4.2]. Als Bewertungsmaßstab für diese Arbeit sind zwei Teilmerkmale des Hauptmerkmals **Maintainability** (Wartbarkeit) relevant.

Modularity beschreibt den Grad, zu dem ein System aus voneinander unabhängigen Komponenten besteht, sodass eine Änderung an einer Komponente möglichst geringe Auswirkungen auf andere Komponenten hat [3, Abschn. 4.2.7.1]. Ein System mit hoher Modularität ermöglicht es, einzelne Teile isoliert zu verstehen, zu testen und auszutauschen, ohne das Gesamtsystem zu destabilisieren.

Modifiability beschreibt den Grad, zu dem ein System effektiv und effizient verändert werden kann, ohne dabei unbeabsichtigte Seiteneffekte in anderen Teilen des Systems zu erzeugen [3, Abschn. 4.2.7.3]. Dieses Teilmerkmal ist eng mit Modularität verknüpft: Ein System mit schlechter Modularität erschwert gezielte Modifikationen, da Änderungen an einer Stelle unkontrolliert auf andere Stellen ausstrahlen.

Beide Teilmerkmale sind unmittelbar mit dem Anspruch der Clean Architecture verknüpft. Die Dependency Rule und die daraus resultierende Schichtentrennung sind präzise darauf

ausgelegt, Modularität und Modifiability zu maximieren. Die Clean Architecture liefert damit nicht nur ein Designprinzip, sondern zugleich den Maßstab, an dem ihre eigene Umsetzung gemessen werden kann.

3

Architektur des Projekts

3.1 Schichtenstruktur

3.2 Stärken der Umsetzung

3.3 Der kritische Befund

4

Analyse und Bewertung

4.1 Warum ist das kritisch?

4.2 Bewertung mit FMEA/RPZ

4.3 Widerspruch zur eigenen Dokumentation

5

Optimierungsmaßnahmen

5.1 Repository-Zugriffe in Controller eliminieren

5.2 Service-Annotation verschieben

5.3 Erwartete Auswirkungen im PDCA-Rahmen

6

Fazit

A Literatur

- [1] L. Briem, „Clean Architecture: Systeme für die Ewigkeit“, 2024.
- [2] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Pearson Education, 2018.
- [3] ISO/IEC, „Systems and software engineering — Systems and software quality requirements and evaluation (SQuaRE) — System and software quality models“, Nr. ISO/IEC25010:2011. 2011.

B Glossar

Selbstständigkeitserklärung

Gemäß Ziffer 1.1.13 der Anlage 1 zu §§ 3, 4 und 5 der Studien- und Prüfungsordnung für die Bachelorstudiengänge im Studienbereich Technik der Dualen Hochschule Baden- Württemberg vom 29.09.2017. Ich versichere hiermit, dass ich meine Arbeit mit dem Thema:

Softwarequalität

selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass alle eingereichten Fassungen übereinstimmen.

Karlsruhe, 19.05.2026

Etienne Luke Josef Bader